Homework: Safe Driving with Deep Q-Networks in MetaDrive

Practical Neural Networks Course
Spring 2026, KNTU
Instructor: Maryam Abdolali



Figure 1: A driving scene from the MetaDrive simulator. In this homework, your DQN agent will learn to make driving decisions in a simulated environment like this.

Objective

In this homework, you will implement a Deep Q-Network (DQN) agent and train it in the `MetaDriveEnv` driving simulator. The goal is to learn a driving policy that achieves high reward while avoiding unsafe behavior such as crashes.

By completing this assignment, you will practice building a Q-network, implementing experience replay, using epsilon-greedy exploration, applying Bellman updates, and evaluating a reinforcement learning agent in a realistic driving environment.

Environment: MetaDrive

This assignment uses the `MetaDriveEnv` environment from the MetaDrive simulator. MetaDrive provides a simulated driving task in which the agent controls a vehicle. At each time step, the environment returns an observation, a reward, termination signal, truncation signal, and an `info` dictionary.

The observation returned by MetaDrive must be converted into a flat state vector before it is passed to the DQN. You should implement or use an observation-processing function such as:

```
state = flatten_obs(obs)
```

The DQN agent chooses from six discrete driving actions. These actions represent combinations of steering and acceleration or braking:

Action Index	Meaning
0	Left + Brake
1	Left + Forward
2	Straight + Brake
3	Straight + Forward
4	Right + Brake
5	Right + Forward

Although the DQN outputs a discrete action index, MetaDrive expects a continuous control action. Therefore, you must convert the selected discrete action into a MetaDrive-compatible action before calling `env.step`. For example, you may implement an action-conversion function such as:

```
action = discrete_to_continuous_action(action_idx)
```

During training, track the total episode reward, episode length, and whether the vehicle crashed. Crash information should be obtained from the `info` dictionary returned by MetaDrive.

Task 1: Create the MetaDrive Environment

Create and configure a `MetaDriveEnv` environment for training. Your configuration should be suitable for a single-agent driving task.

Your environment setup should include the parameters needed to run training reproducibly, such as the number of maps, random seed, rendering mode, and whether crash information is available through the `info` dictionary.

You should verify that the environment can be reset and stepped through before training the DQN agent.

Task 2: Implement Observation and Action Processing

Implement a function that converts MetaDrive observations into flat state vectors that can be used as input to a neural network.

You should also implement a function that maps the six discrete DQN actions to MetaDrive-compatible continuous control actions. The action should include steering and acceleration or braking information.

Task 3: Implement the DQN Neural Network

Implement the `DQNNetwork` class.

The network should take a flattened MetaDrive observation as input and output one Q-value for each of the six discrete driving actions. Use two hidden fully connected layers with ReLU activations. The architecture should be:

$$\texttt{state_size} \to \texttt{hidden_size} \to \texttt{hidden_size} \to \texttt{action_size}$$

The output of the network should represent:

$$Q(s, a_0), Q(s, a_1), \ldots, Q(s, a_5)$$

where each value estimates how good it is to take a particular action in state s.

Task 4: Implement the Experience Replay Buffer

Implement the `ReplayBuffer` class.

The replay buffer should store transitions of the form:

$$(s_t, a_t, r_t, s_{t+1}, d_t)$$

Here, s_t is the current flattened MetaDrive state, a_t is the selected discrete action, r_t is the reward, s_{t+1} is the next flattened state, and d_t indicates whether the episode ended.

Your replay buffer should be able to add new transitions, randomly sample mini-batches, return the sampled data as PyTorch tensors, and report how many transitions are currently stored.

Task 5: Implement the DQN Agent

Implement the `DQNAgent` class.

The agent should contain a Q-network, a target network, an optimizer, and a replay buffer. It should select actions using an epsilon-greedy policy, learn from sampled replay-buffer batches, and periodically update the target network.

During training, the agent should choose a random action with probability ϵ and choose the greedy action with probability $1 - \epsilon$:

$$a = \begin{cases} \text{random action}, & \text{with probability } \epsilon, \\ \arg\max_a Q(s, a), & \text{with probability } 1 - \epsilon. \end{cases}$$

The DQN target should be computed using the Bellman update:

$$y = r + (1 - d)\gamma \max_{a'} Q_{\text{target}}(s', a')$$

where $d = 1$ if the episode ended and $d = 0$ otherwise.

Train the Q-network using mean squared error loss:

$$\mathcal{L} = (Q(s, a) - y)^2$$

Task 6: Implement the Training Loop

Implement the training loop for the DQN agent.

At the start of each episode, reset the MetaDrive environment and flatten the initial observation. At each step, use the DQN agent to select a discrete action, convert that action into the format expected by MetaDrive, step the environment, flatten the next observation, and store the transition in the replay buffer.

The agent should begin learning only after enough experiences have been collected. The episode should end when MetaDrive reports either termination or truncation. For every episode, record the total reward, the number of steps, and whether a crash occurred.

Task 7: Train the Agent

First train the agent for a small number of episodes, such as 5–20, to make sure your implementation runs correctly. After debugging, train for a larger number of episodes, such as 500–1000, for the final experiment.

After training, save the trained model as:

`models/dqn_trained.pt`

Task 8: Evaluation and Visualization

Plot the training results and evaluate the trained policy. Your evaluation should discuss whether the reward improves over time, whether the episode length changes, and whether the crash rate decreases.

Visualize the trained agent in MetaDrive using both a 3D view and a top-down view. Report the average reward and crash behavior from the evaluation episodes.

Deliverables

Submit your completed notebook or Python files, the training plots, the saved model file, and your answers to the reflection questions. The saved model should be named:

`models/dqn_trained.pt`

Your code should run from start to finish without errors.